התחיל ב:	10/08/2023, 17:49
מצב	הסתיים
הושלם ב-	10/08/2023, 17:49
הזמן שלקח	11 שניות

שאלה **1**

לא נענה

ניקוד השאלה: 5.00

מה סיבוכיות זמן הריצה של **f** כפונקציה של **n**?

```
def f(n):
    i, s = 1, 0
    while i <= n:
        s += g(n)
        i += 1
    return s

def g(n):
    i, s = 1, 0
    while i <= n:
        s += i**3
        i += 1
    return s
```

שאלה **2**

לא נענה

ניקוד השאלה: 5.00

השלימו במקומות המסומנים את קוד הגנרטור **break_it(f, x)** אשר מקבל פונקציה **f** וערך התחלתי **x**. הערך שיתקבל אחרי שנפעיל **k** פעמים את **next**, הוא תוצאת הפעלה **k-1** פעמים את הפונקציה **f** על **x**. (כלומר, הערך הראשון שמוחזר הוא **x** עצמו.) הגנרטור יפסיק כאשר ערכו הבוליאני של הערך הבא שהיה יוצא יהיה **False**, כמו **0, [], False, ""**. אם ערכו הבוליאני של **x** עצמו הוא **False**, הגנרטור לא יוציא אף ערך.

```
def break_it(f, x):
    while ______(1)______:
        yield ______(2)______
        ______(3)______

assert (list(break_it(lambda s: s [1:], 'exams'))==
        ['exams', 'xams', 'ams', 'ms', 's'])
assert (list(break_it(lambda s: s[1::2], 'generator'))==
        ['generator', 'eeao', 'eo', 'o'])
assert (list(break_it(lambda x: x//2-1, 36))==
        [36, 17, 7, 2])
```

(1)

(2)

(3)

שאלה **3**

לא נענה

ניקוד השאלה: 5.00

מה סיבוכיות זמן הריצה של הפונקציה **sum_to_n_count** אשר מקבלת רשימה **lst** של שלמים חיוביים, ומספר שלם חיובי **n**, כפונקציה של **n** ושל **s=len(lst)**?

```
def sum_to_n_count(lst,n):
    vals = [1]+[0]*n
    for i in lst:
        for j in reversed(range(n-i+1)):
            vals[j+i]+=vals[j]
    return vals[n]
```

O(

)

שאלה **4**

לא נענה

ניקוד השאלה: 5.00

השלימו במקומות המסומנים את קוד הפונקציה **tree_cut(t, k)** אשר מקבלת עץ **t** ומספר שלם אי שלילי **k**, וגוזמת את הענפים האחרונים (לפי סדר הופעתם) מכל קדקד, כך שלאף קדקד בעץ לא יהיו יותר מ-**k** ענפים. אין גיזום ענפים לקדקדים שיש להם **k** ענפים או פחות. הפונקציה תשנה את העץ המקורי. אין דרישה לערך החזרה ספציפי.

```
class Tree:
    def __init__(self,label,branches):
        self.label=label
        self.branches=list(branches)

    def __eq__(self,other):
        ... #RETRACTED


def tree_cut(t,k):
    ______(1)______ = ______(2)______
    return ______(3)______

t = Tree(10,
         [Tree(1), Tree(2), Tree(3),
          Tree(4), Tree(5), Tree(6)])
t2 = Tree(10,
         [Tree(1), Tree(2), Tree(3),
          Tree(4), Tree(5), Tree(6)])
assert tree_cut(t,8)==t2
assert tree_cut(t,6)==t2
assert tree_cut(t,2)==Tree(10,[Tree(1),Tree(2)])
assert tree_cut(t,0)==Tree(10)
assert tree_cut(t,6)==Tree(10)
```

(1)

(2)

(3)

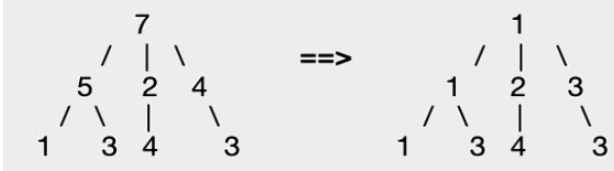
שאלה **5**

לא נענה

ניקוד השאלה: 5.00

הורד משתנה k מחתימת הפונקציה בתמונה. הדוגמה הוחזרה.

השלימו במקומות המסומנים את קוד הפונקציה **get_min_tree(t)** אשר מקבלת עץ **t**, ומחזירה עץ חדש כך שהמספר בכל קדקד הוא הקטן ביותר שנמצא בתתי העצים שלו (כולל בקדקד עצמו).




```
class Tree:
    def __init__(self,label,branches):
        self.label=label
        self.branches=list(branches)

def get_min_tree(t):
    var = ______(1)______
    return ______(2)______
```

(1)

(2)

שאלה **6**

לא נענה

ניקוד השאלה: 10.00

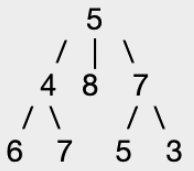
השלימו במקומות המסומנים את קוד הגנרטור **gen(t, n)** אשר מקבל עץ **t** ומספר שלם **n**, ומחזיר את כל ערכי הזוגות אב-בן מבין קדקדי העץ **t**, שסכום ערכיהם הוא **n**. כל זוג כזה מוגש ארוז ברשימה של שני איברים.

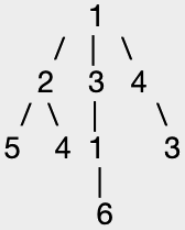
```
class Tree:
    def __init__(self,label,branches):
        self.label=label
        self.branches=list(branches)

def gen(t, n):
    for b in ______(1)______:
        if ______(2)______ == n:
            ______(3)______
        yield from ______(4)______

t = Tree(5,[Tree(4,[Tree(6),Tree(7)]),
            Tree(8),
            Tree(7,[Tree(5), Tree(3)])])
assert list(gen(t,10))==[[4,6],[7,3]]
assert list(gen(t,9))==[[5,4]]
assert list(gen(t,7))==[]

t = Tree(1, [Tree(2, [Tree(5), Tree(4)]),
             Tree(3, [Tree(1, [Tree(6)])]),
             Tree(4, [Tree(3)])])
assert list(gen(t,7))==[[2,5],[1,6],[4,3]]
assert list(gen(t,4))==[[1,3],[3,1]]
assert list(gen(t,8))==[]
```





(1)

(2)

(3)

(4)

שאלה **7**

לא נענה

ניקוד השאלה: 5.00

השלימו במקומות המסומנים את קוד הפונקציה **sub_lists(lst)** אשר מקבלת רשימה **lst** ומשתמשת בפונקציה **inject_each** כדי להחזיר רשימה של כל תתי הרשימות של **lst**.

```
def inject_each(x, nest_lis):
    return [[x] + lst for lst in nest_lis]

assert inject_each(10,[[],[1],[2,3]])==[[10],[10,1],[10,2,3]]

def sub_lists(lst):
    if not lst:
        return ______(1)______
    else:
        sub = ______(2)______
        return ______(3)______

assert sub_lists([1])==[[1],[]]
assert sub_lists([1,2])==[[1,2],[1],[2],[]]
assert sub_lists([1,2,3])==[[1,2,3],[1,2],[1,3],[1],
                            [2,3],[2],[3],[]]
```

(1)

(2)

(3)

שאלה **8**

לא נענה

ניקוד השאלה: 15.00

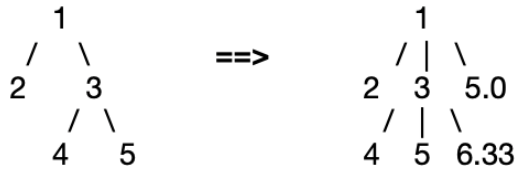
נתונה המחלקה **Tree** הבאה. הניחו שבקדקודים של העצים (**labels**) נמצאים מספרים.

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

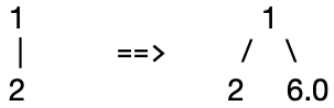
    def __repr__(self):
        if self.branches:
            return f'Tree({self.label}, {self.branches})'
        return f'Tree({self.label})'
```

כתבו את הפונקציה **insert_average(t, val)** אשר מקבלת עץ **t** ומספר שלם **val**, ולכל קדקד **d** בעץ שאינו עלה, מוסיפה ילד אחד חדש שערכו יהיה הממוצע של **val** עם כל ערכי הילדים (ולא שאר הצאצאים) הקיימים של **d**. הפונקציה תחזיר עץ חדש ולא תשנה את העץ המקורי. דוגמאות:

```
>>> t = Tree(1, [Tree(2), Tree(3, [Tree(4), Tree(5)])])
>>> insert_average(t, 10)
Tree(1, [Tree(2), Tree(3, [Tree(4), Tree(5), Tree(6.333333333333333)]), Tree(5.0)])
```




```
>>> insert_average(Tree(1, [Tree(2)]),10)
Tree(1, [Tree(2), Tree(6.0)])
```



Answer:

(penalty regime: 0 %)

Reset answer

```
ranches=[]):

anches)


.label}, {self.branches})'
el})'

INE!
```

שאלה **9**

לא נענה

ניקוד השאלה: 15.00

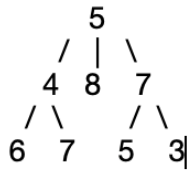
נתונה המחלקה **Tree** הבאה. הניחו שבקדקודים של העצים (**labels**) נמצאים מספרים.

```
class Tree:
    def __init__(self, label, branches=[]):
        self.label = label
        self.branches = list(branches)

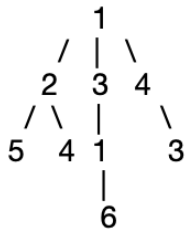
    def __repr__(self):
        if self.branches:
            return f'Tree({self.label}, {self.branches})'
        return f'Tree({self.label})'
```

כתבו את הפונקציה **long_ways(t)** אשר מקבלת עץ **t**, ומחזירה את מספר המסלולים מהשורש ועד לעלה בעץ שבהם כל הערכים גדולים או שווים מהערך שבשורש. דוגמאות:

```
>>> t = Tree(5, [Tree(4, [Tree(6), Tree(7)]), Tree(8), Tree(7, [Tree(5), Tree(3)])])
>>> long_ways(t)
2 # 5-8, 5-7-5
```




```
>>> t = Tree(1, [Tree(2, [Tree(5), Tree(4)]), Tree(3, [Tree(1, [Tree(6)])]), Tree(4, [Tree(3)])])
>>> long_ways(t)
4 # 1-2-5, 1-2-4, 1-3-1-6, 1-4-3
```




```
>>> t = Tree(7)
>>> long_ways(t)
1 # 7
```

Answer:

(penalty regime: 0 %)

Reset answer

```
ranches=[]):

anches)


.label}, {self.branches})'
el})'

INE!
```

שאלה **10**

לא נענה

ניקוד השאלה: 15.00

נתונה המחלקה **Link** הבאה:

```
class Link:
    def __init__(self, value, next=None):
        self.value = value
        self.next = next

    def __repr__(self):
        if self.next:
            return f'{self.value}->{self.next}'
        return f'{self.value}'
```

כתבו את הפונקציה **inter_link_list(lnk, lst)** אשר מקבלת רשימה מקושרת **lnk** ורשימה **lst**. גם ערכי הצמתים של **lnk** וגם אברי הרשימה **lst** הם שלמים אי-שליליים. הפונקציה מחזירה את **lnk** עצמה כאשר בין הצמתים שלה משלבים צמתים חדשים בהם איברים מהרשימה **lst** באופן הבא:

- בין כל שני צמתים עוקבים של **lnk**, שערכיהם זוגיים, תיכנס צומת חדשה עם מספר אי זוגי מ **lst**,
- בין כל שני צמתים עוקבים של **lnk**, שערכיהם אי-זוגיים תיכנס צומת חדשה עם מספר זוגי מ **lst**.

אם אין יותר מספרים מתאימים כדי לשלב ב **lnk**, או שהפונקציה סיימה לעבור על כל הצמתים ב **lnk** וב **lst** נשארו מספרים שלא שולבו: **lnk** תוחזר במצב אליו הגיעה.

את זוגות הצמתים של **lnk**, ואת אברי הרשימה **lst** "מבקרים" לפי סדר הופעתם:

```
>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst1 = [12,2,31,7]
>>> inter_link_list(lnk1, lst1)
1->12->3->2->5->4->31->6->7->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst2 = [1]
>>> inter_link_list(lnk1, lst2)
1->3->5->4->1->6->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
```

```
>>> lst3 = [11,3]
>inter_link_list(lnk1, lst3)
1->3->5->4->11->6->3->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst4 = [22,44]
>>> inter_link_list(lnk1, lst4)
1->22->3->44->5->4->6->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst5 = [1,1]
>>> inter_link_list(lnk1, lst5)
1->3->5->4->1->6->1->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst6 = []
>>> inter_link_list(lnk1, lst6)
1->3->5->4->6->8

>>> lnk1 = Link(1,(Link(3,Link(5,Link(4,Link(6,Link(8)))))))
>>> lst7 = list(range(20))
>>> inter_link_list(lnk1,lst7)
1->0->3->2->5->4->1->6->3->8
```

Answer:

(penalty regime: 0 %)

Reset answer

```
ext=None):



e}->{self.next}'

INE!
```

שאלה **11**

לא נענה

ניקוד השאלה: 15.00

כתבו גנרטור **fly_filter(indices, values)** אשר מקבל שני איטרטורים. הערכים שיוצאים מהאיטרטור הראשון (**indices**) הם מספרים שלמים אי-שליליים, ובסדר עולה.

הגנרטור עובר לפי הסדר על אברי **indices** כך שעבור אבר מ-**indices** שערכו **n**, הגנרטור יחזיר (**yield**) את האיבר במקום ה-**n** מתוך **values**. אם אין יותר איברים מתאימים באחד האיטרטורים, הגנרטור יסיים את ריצתו.

בכתיבת הגנרטור אין לגשת לאברי הקלטים באמצעות אינדקסים (שלא קיימים) ואין להשתמש בפונקציות המרה כמו **list, str** על הקלטים. דוגמאות:

```
>>> a=iter([100])
>>> b=iter("324567190")
>>> list(fly_filter(a,b))
[]
>>>
>>> a={}
>>> b="324567190"
>>>
list(fly_filter(iter(a),iter(b)))
[]
>>>
>>> a=iter(())
>>> b=iter(())
>>> list(fly_filter(a,b))
[]
```

```
>>> a={1:'x',3:'y',5:'z',
      6:'w',9:'r',11:"D"}
>>> b="324567190"
>>>
list(fly_filter(iter(a),iter(b)))
['2', '5', '7', '1']
>>>
>>> a=iter((1,))
>>> b=iter([100,200])
>>> list(fly_filter(a,b))
[200]
>>>
>>> a=iter([1,4,5,7])
>>> b=iter([10,20,30,40,50,1,2,0])
>>> list(fly_filter(a,b))
[20, 50, 1, 0]
```

```
>>>
a=iter({0:'x',2:'y',4:'z',6:'w'})
>>> b=iter("abcdefghijk")
>>> list(fly_filter(a,b))
['a', 'c', 'e', 'g']
>>>
>>> a=
{3:'y',5:'z',6:'w',9:'r',11:"D"}
>>> b=
{1:'q',3:'f',4:'v',6:'m',9:"Z"}
>>>
list(fly_filter(iter(a),iter(b)))
[6]
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **12**

לא נענה

ניקוד השאלה: 15.00

השלימו את הקוד עבור השיטות **__repr__, free, record, spy** במחלקה **Eyes** הבאה:

```
class Eyes:
    def __init__(self):
        self.seen = []

    def spy(self, it1):
        ...

    def record(self, it2):
        ...

    def free(self, it3):
        ...

    def __repr__(self):
        ...
```

השיטה **spy** מקבלת iterable כלשהו למשתנה **it1** ומוסיפה לפי הסדר את איבריו לסוף הרשימה **seen** אשר מאותחלת בשיטה **__init__**.

השיטה **record** מקבלת iterable בשם **it2**, ומחזירה מילון בו המפתחות הם כל אברי **it2** והערכים הם מספר המופעים שלהם ב **seen**.

השיטה **free** מקבלת iterable בשם **it3**, ומוחקת מהרשימה **seen** את כל המופעים של כל אחד מאברי **it3**.

השיטה **__repr__** מחזירה ייצוג מחרוזת של אובייקט מהמחלקה **Eyes**.

- הניחו שאברי **it1, it2, it3** הינם hashable. אין לשנות את השיטה **__init__**, ואין להוסיף שדות למחלקה. דוגמאות:

```
>>> s=Eyes()
>>> s
[]
>>> s.spy((2, 3, 4))
>>> s
[2, 3, 4]
>>> s.record([3, 4])
{3: 1, 4: 1}
>>> s.spy(range(1,4))
>>> s
[2, 3, 4, 1, 2, 3]
>>> s.record([3, 4])
{3: 2, 4: 1}
>>> s.record(['a', 'b', 1])
{'a': 0, 'b': 0, 1: 1}
>>> s.record(s.seen)
{2:2, 3:2, 4:1, 1:1}
>>> s.spy("ba"}
>>> s
[2, 3, 4, 1, 2, 3, 'b', 'a']
>>> s.free([1,3,'c',4])
>>> s
[2, 2, 'b', 'a']
>>> s.record(['a',2,3,'c',4,'a',4])
{'a': 1, 2: 2, 3: 0, 'c': 0, 4: 0}
```

Answer:

(penalty regime: 0 %)

Reset answer

```
IS LINE!
```

שאלה **13**

לא נענה

ניקוד השאלה: 15.00

עבור **k** חיובי, נגדיר את ההיקף ה-**k**-י של מטריצה ריבועית להיות מסלול שעובר בשורות: **k-1** ,**n-k** ובעמודות **n-k** ,**k-1**.

כתבו את הפונקציה **perimatrix(n, k)** אשר מקבלת שני מספרים שלמים המקיימים **n>=k>0//2(n+1)** ויוצרת מטריצה בגודל **n*n** של אפסים, שבה לאורך מסלול ההיקף ה-**k**-י יכתבו המספרים מ-1 עד **4*(1+k-2*n)** לפי סדר(ורק את **1** במקרה הקצה). המספרים יתחילו בפינה השמאלית העליונה וימשיכו עם כיוון השעון. דוגמאות:

```
>>> from pprint import pprint
>>> pprint(perimatrix(5,1))
[[1, 2, 3, 4, 5],
 [16, 0, 0, 0, 6],
 [15, 0, 0, 0, 7],
 [14, 0, 0, 0, 8],
 [13, 12, 11, 10, 9]]



>>> pprint(perimatrix(6,1))
[[1, 2, 3, 4, 5, 6],
 [20, 0, 0, 0, 0, 7],
 [19, 0, 0, 0, 0, 8],
 [18, 0, 0, 0, 0, 9],
 [17, 0, 0, 0, 0, 10],
 [16, 15, 14, 13, 12, 11]]
```

```
>>> pprint(perimatrix(5,2))
[[0, 0, 0, 0, 0],
 [0, 1, 2, 3, 0],
 [0, 8, 0, 4, 0],
 [0, 7, 6, 5, 0],
 [0, 0, 0, 0, 0]]



>>> pprint(perimatrix(6,2))
[[0, 0, 0, 0, 0, 0],
 [0, 1, 2, 3, 4, 0],
 [0, 12, 0, 0, 5, 0],
 [0, 11, 0, 0, 6, 0],
 [0, 10, 9, 8, 7, 0],
 [0, 0, 0, 0, 0, 0]]
```

```
>>> pprint(perimatrix(5,3))
[[0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 1, 0, 0],
 [0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0]]



>>> pprint(perimatrix(6,3))
[[0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0],
 [0, 0, 1, 2, 0, 0],
 [0, 0, 4, 3, 0, 0],
 [0, 0, 0, 0, 0, 0],
 [0, 0, 0, 0, 0, 0]]
```

Answer:

(penalty regime: 0 %)

Reset answer

שאלה **14**

לא נענה

ניקוד השאלה: 1.00

סמנו אלו שאלות בחרתם בחלק ב', השאלות שתסמנו הן אלו שיבדקו **בלבד**.

שאלה 8 - average

- שאלה 9 - longways
- שאלה 10 - interlink
- שאלה 11 - flyfilter
- שאלה 12 - eyes
- שאלה 13 - perimatrix

שאלה **15**
לא נענה
לא ניתן ציון

תוכלו לרשום כאן הערות וטקסט חופשי במידת הצורך. **חלק זה אינו נבדק מראש** אלא רק במקרים חריגים ובמקרה של ערעורים.